iPhone 6 Plus Android Bring-up Report

Date: May 20, 2026

Device: iPhone 6 Plus, iPhone7,1 / N56AP / Apple A8 T7000, iOS 12.5.8

Goal

The goal is to get a real Android/Linux-based system running on the iPhone 6 Plus, not just to boot a diagnostic shell. A Pixel-style Android userspace or Android GSI is only useful after the Linux kernel can see the phone hardware, especially display, USB, input, and internal storage.

Short Summary

We successfully booted custom Linux payloads on the iPhone 6 Plus through DFU, PongoOS, m1n1, and Hoolock Linux. The phone can run Linux code and expose a USB shell. We added a device-tree node and a Linux diagnostic driver for the A8 ANS storage controller path, then tested it on the physical phone.

The big result is that Linux can power and talk to the ANS hardware region, but the internal NAND does not appear as a block device yet. This is not because Android is the wrong image and not because APFS is unreadable. Linux currently lacks the A8-era Apple ANS / RTBuddy / ASP storage driver needed to turn the iPhone NAND into something like /dev/sda, /dev/mmcblk0, or an Apple NAND block device.

In plain terms: the phone is telling us the blocker is the Apple storage controller protocol, not the physical NAND package. iOS has an ANS/RTBuddy/ASP storage stack, and Linux does not yet have the A8 driver for that path.

The most useful breakthrough was discovering that A8 does not use the newer ASC/RTKit mailbox path for ANS. It uses the older AKF mailbox path at ANS base plus 0x1000. After adding AKF mailbox diagnostics, the phone returned real ANS/RTBuddy management messages.

The latest confirmed phone-side milestone is that Linux can stage the recovered old RTBuddy packet shapes from iOS 12.5.8 and run guarded packet-address candidate tests against the live controller. Those tests did not expose NAND, but they ruled out another major wrong path: the missing transaction is not a plain physical address, shifted address, or simple address/length pair sent through the AKF mailbox.

What We Accomplished

1. Booted custom Linux on the iPhone 6 Plus through DFU and PongoOS.

2. Confirmed the device identity from Linux:

   - Apple iPhone 6 Plus
   - apple,n56
   - apple,t7000
   - Apple A8 / T7000

3. Confirmed the iOS SSH ramdisk can see the internal storage:

- ASPStorage
- ASPBlockStorage
- IONANDBlockDevice
- Apple NAND Media
- disk0
- 128 GB Toshiba NAND
- 4096-byte block size
- valid GPT protective MBR ending in 55 aa

4. Confirmed Hoolock Linux does not see internal storage yet:

   - Linux sees RAM disks.
   - Linux sees small MTD entries for boot artifacts.
   - Linux does not expose the internal iPhone NAND as a usable block device.

5. Added the A8 ANS node to the T7000 device tree:

   - physical address 0x208040000
   - length 0x2000
   - power domain ps_ans
   - interrupts matching the iOS ADT data

6. Added a Linux diagnostic driver:

   - drivers/soc/apple/t7000-ans-probe.c
   - bound to /soc/ans@208040000
   - exposes ANS registers through sysfs
   - exposes ASC and AKF mailbox diagnostics

7. Proved Linux can power and map the ANS MMIO region:

   ‘text
   mmio_size=0x2000 pm_ret=0 runtime_status=active
   ‘

8. Tested the newer ASC/RTKit-style mailbox path.

   Result: it did not produce a HELLO on A8. The CPU start bit did not behave like newer Apple Silicon ANS controllers.

9. Found the correct A8-era mailbox family in local m1n1 source:

   - hoolock-m1n1/src/akf.c
   - iop,s5l8960x uses AKF
   - AKF mailbox base is ANS base plus 0x1000
   - messages are split into 32-bit halves

10. Tested AKF on the real phone and got live ANS/RTBuddy traffic.

11. Added an old RTBuddy state-machine view:

    - oldrtbuddy_state
    - tracks the four observed management phases
    - confirms the loop is stable
    - records the last message and mailbox control bits

12. Added an ordered reply path:

    - oldrtbuddy_plan
    - oldrtbuddy_ordered_reply
    - refuses to send unless the controller is in the expected phase
    - intended for controlled ANS/RTBuddy acknowledgement experiments

13. Booted the ordered-reply build on the phone and confirmed the new path is live.
14. Recovered two old RTBuddy packet layouts from the iOS 12.5.8 RTBuddyManagementEndpoint code path:

    - old-small: prefix 0400000054000000, word0=0x6, word1=0x11, total length 0x54
    - old-0x40: prefix 0400000054000000, word0=0x7, word1=0x44, total length 0x120

15. Added guarded packet staging and packet-address candidate sysfs paths:

    - oldrtbuddy_packet_model
    - oldrtbuddy_packet_stage
    - oldrtbuddy_packet_dump
    - oldrtbuddy_packet_addr_plan
    - oldrtbuddy_packet_addr_send

16. Booted kernel #17 on the phone and tested packet-address candidates in one pass. The controller stayed alive and stable, but did not advance to endpoint discovery or NAND block-device registration.

Important Phone-side Evidence

The AKF mailbox exposed live values:

```
akf_set          @1000: 00001111
akf_clr          @1004: 00001111
akf_a2i_control  @1008: 00023301
akf_a2i_send0    @1010: 00000001
akf_a2i_send1    @1014: 00600000
akf_a2i_recv0    @1018: 000ff013
akf_a2i_recv1    @101c: 06000000
akf_i2a_control  @1020: 00020001
akf_i2a_recv0    @1038: 00000012
akf_i2a_recv1    @103c: 06000000
```

Polling the AKF receive side produced a repeating management sequence:

```
0600000000000012
0600000000000004
00b0000000000010
0070000000000001
```

That means the ANS storage controller is not dead and not invisible. Linux is now reaching the correct mailbox family. The remaining task is to decode and respond to this older RTBuddy management sequence, then start ANSEndpoint1.

The ordered state-machine path was later confirmed on the phone:

```
oldrtbuddy_ordered_reply=v1
stable=1 samples=64 unknown=0 phase=stable-management-loop
last_msg=0070000000000001 loop_count=7 seen_mask=f
```

The driver also confirmed the four expected phases:

```
state12     expects 0600000000000012
state04     expects 0600000000000004
ap_power10  expects 00b0000000000010
iop_ack01   expects 0070000000000001
```

This is important because it moves the work from one-off guessed writes to a controlled protocol experiment. The driver can now refuse to send a reply if the ANS controller is not at the matching phase of the old RTBuddy management loop.

The later packet-address candidate run tested whether old RTBuddy simply wanted the staged packet buffer address through the mailbox. The tested messages were:

```
old-small addr64          -> 000000080543a000
old-0x40 addr64           -> 000000080543a000
old-0x40 addr-shift4      -> 0000000080543a00
old-0x40 addr-shift12     -> 000000000080543a
old-0x40 addr-len         -> 000001200543a000
old-0x40 len-addr         -> 0543a00000000120
```

All writes returned success from the guarded Linux path, but the controller remained in the stable management loop. The safest interpretation is that the traffic reached the right layer but was incomplete, malformed, structurally insufficient, or unrelated to the expected transaction flow. It was not fatal; it was just not valid enough to advance.

The current Linux partition list is still:

```
ram0..ram15
mtdblock0
mtdblock1
```

The tiny MTD devices are not the internal NAND. They are only 16 KB and 128 KB, which matches boot/test artifacts such as logs or the device tree. The iPhone NAND is about 128 GB, so it would appear as a much larger block device after the ANS/RTBuddy/ASP stack is initialized.

Why Android Still Does Not Boot

Android needs a Linux kernel with drivers for the hardware it is running on. Right now, the kernel can boot, display logs, and provide a USB shell, but it cannot mount or install to the internal iPhone storage because the internal storage block device is not registered.

The storage path on iOS is:

```
ans@8040000
  iop-ans-nub
    RTBuddy
      ANSEndpoint1
        ASPStorage
          ASPBlockStorage
            IONANDBlockDevice
              Apple NAND Media -> disk0
```

Linux needs an equivalent path:

```
ANS MMIO
  AKF mailbox
    old RTBuddy management protocol
      ANSEndpoint1
        ASPStorage / ASPBlockStorage commands
          Linux block device
```

Until that exists, Android images, GSIs, APFS support, and ext4 filesystems do

not solve the storage problem. They come after the block device exists.

Hardware / Schematic Note

The iPhone 6 Plus schematic identifies the NAND package as U0604, with Apple part variants for 16 GB, 64 GB, and 128 GB NAND from vendors such as Hynix and Toshiba. The relevant signal names include:

```
AP_BI_NAND_ANC0_IO<0..7>
AP_BI_NAND_ANC1_IO<0..7>
AP_TO_NAND_ANC0_CLE
AP_TO_NAND_ANC0_ALE
AP_TO_NAND_ANC0_WE_L
AP_TO_NAND_ANC0_RE_L
NAND_TO_PP_RB
PP3V0_NAND
PP1V2_NAND_VDDI
```

That supports the current software finding: the raw NAND package is not the main blocker. The package sits behind Appleâ s A8-era NAND/ANS/ASP controller path. Replacing or blanking the NAND would not automatically make Linux see storage unless Linux can speak the old RTBuddy / ANS / ASPStorage protocol.

The A8 family comparison is useful too. The Apple A8/T7000 was used across iPhone 6, iPhone 6 Plus, iPad mini 4, Apple TV HD, HomePod, and iPod touch 6, and iPad mini 4 teardown material also shows an A8 plus external NAND package. So the physical architecture is plausible for Linux/Android, but the custom Apple controller stack is still the real engineering boundary.

About The NAND Theory

It is understandable to suspect the NAND chip because the failure looks like "Linux cannot see the storage." But the evidence so far does not show that the physical NAND chip has a special block that prevents Android.

What the evidence shows:

- iOS can see and read the NAND.
- The iOS storage stack talks through Apple-specific layers: ANS, RTBuddy, ASPStorage, ASPBlockStorage, and IONANDBlockDevice.
- Linux does not yet implement the A8 version of that stack.
- Once we used the correct AKF mailbox family, ANS started returning real management messages.

So the phone is not showing us a dead, locked, or unusable NAND chip. It is showing us a storage controller protocol gap: Apple already has the ANS/RTBuddy/ASP stack in iOS, while Linux still needs an A8-compatible driver for that path.

That points to a missing software driver/protocol implementation, not a NAND chip that must be physically replaced.

Desoldering the NAND and installing a blank one would probably not make Android installable by itself. The A8 SoC and boot chain would still need to initialize ANS, speak RTBuddy, speak ASPStorage, and expose a block device. A blank NAND could also create new boot, calibration, pairing, or restore problems.

A better next path is to continue implementing the ANS/RTBuddy/ASP driver. If

that driver eventually exposes the NAND as a Linux block device, then we can decide whether to mount existing partitions read-only, create an Android data area, or boot Android from external/ramdisk storage first.

What The Next Engineering Step Is

The next practical step is not another Android image test. It is implementing a controlled AKF receive/respond path:

1. Decode the old A8 RTBuddy management messages:

   ‘text
   0600000000000012
   0600000000000004
   00b0000000000010
   0070000000000001
   ‘

2. Add safe code to acknowledge the messages instead of only polling them.
3. Use the new ordered-reply path to test a specific acknowledgement sequence only when the controller is at the matching phase.
4. Identify the endpoint map and find/start ANSEndpoint1.
5. Implement minimal ASPStorage / ASPBlockStorage commands.
6. Register a read-only Linux block device first.
7. Only after that, attempt Android userspace or an Android system image.

Current Status

We have moved from "the phone boots Linux but storage is missing" to "Linux can talk to the correct A8 ANS mailbox, receive storage-controller management messages, classify the stable old RTBuddy loop, and expose a guarded ordered reply path." That is real progress toward Android, but the storage driver is still incomplete.

The phone could plausibly run Android because the CPU is ARM64 and Linux already boots. The blocker is Apple’s custom hardware interface for storage and other peripherals, not the basic ability of the phone to execute Android/Linux code.